

AI ARCHITECT

Practical Interview Questions & Answers

Real-world, architecture-level preparation - 2026 edition

This guide collects the practical, architecture-level questions currently being asked in AI Architect interviews, together with answers written the way a strong candidate would actually deliver them: opinionated, grounded in trade-offs, and tied to production reality rather than textbook definitions.

How to use it

- Read the answer, then close the page and say it out loud in your own words. Recall beats recognition.
- For every answer, be ready with one concrete example from your own work - interviewers probe the second layer.
- Sections 12 and 13 (system design and trade-offs) carry the most weight at architect level.
- The rapid-fire section in Section 15 is for the day before the interview.

15 Sections	87 Questions	Practical Focus
-----------------------	------------------------	---------------------------

Contents

1. LLM Foundations & Model Selection	9 Q&A
2. Prompt & Context Engineering	6 Q&A
3. RAG Architecture	8 Q&A
4. Embeddings & Vector Stores	4 Q&A
5. Fine-tuning, Adaptation & Training	6 Q&A
6. Agentic AI & Multi-Agent Systems	9 Q&A
7. LLMOps, Deployment & Infrastructure	7 Q&A
8. Evaluation, Observability & Quality	5 Q&A
9. Security, Governance & Responsible AI	6 Q&A
10. AI for Quality Engineering & Testing	5 Q&A
11. Cloud, Data & Integration Architecture	4 Q&A
12. System Design Case Studies	6 Q&A
13. Architecture Trade-off Questions	4 Q&A
14. Leadership, Stakeholders & Behavioural	6 Q&A
15. Rapid-Fire Round	2 Q&A

1. LLM Foundations & Model Selection

Q1. What actually happens, step by step, when a prompt hits an LLM in production?

The request is tokenised, embedded, and run through the transformer stack in two distinct phases.

- Prefill: the whole prompt is processed in parallel; this is compute-bound and drives Time To First Token (TTFT).
- Decode: tokens are generated one at a time, each attending to the KV cache; this is memory-bandwidth-bound and drives tokens/sec.
- Architecturally this matters because prefill and decode scale differently. Long prompts hurt TTFT; long outputs hurt throughput. Batching, KV-cache reuse and prompt caching are your main levers.

In an interview, always tie this back to a cost or latency decision - that is what separates an architect answer from a developer answer.

Q2. How do you choose between a frontier model, a mid-tier model, and a small open-weight model?

Drive it from requirements, not preference. I use a four-axis scorecard:

- Task complexity: multi-step reasoning, long-context synthesis and agentic tool use justify frontier models. Classification, extraction, routing and summarisation usually do not.
- Latency budget: sub-second interactive paths push toward small models or cached/distilled variants.
- Data residency and control: regulated or air-gapped workloads push toward self-hosted open weights (Llama, Qwen, Mistral).
- Unit economics: cost per resolved request, not cost per token. A cheap model that needs three retries is more expensive than one good call.

The practical pattern I deploy is a cascade: a small model handles 70-80% of traffic, a router escalates hard cases to a frontier model, and everything is measured against a golden evaluation set.

Q3. What is the difference between context window size and effective usable context?

A model may advertise 200K tokens but degrade well before that. Attention is not uniform - recall drops in the middle of very long inputs (the 'lost in the middle' effect).

- Design for the effective window: put instructions at the top, the task restated at the bottom, and retrieved evidence in between with clear delimiters.
- Prefer retrieval and compression over stuffing. 8K of relevant context beats 180K of loosely related context.
- Long context also costs money and TTFT on every single call, whereas retrieval costs are amortised.

Q4. Explain temperature, top-p, top-k and when you change them in a real system.

- Temperature scales the logit distribution. Low (0-0.2) for extraction, classification, SQL, code and anything evaluated against a ground truth. Higher (0.7-1.0) for ideation and creative copy.
- Top-p (nucleus) keeps the smallest set of tokens whose cumulative probability exceeds p. It adapts better than top-k to varying distributions.
- Top-k keeps a fixed number of candidates - blunter, mostly legacy.

Rule of thumb I give teams: tune temperature OR top-p, not both. And pin them in config, never leave them to defaults, because a silent default change is an untraceable production regression.

Q5. What are MoE models and why does an architect care?

Mixture-of-Experts routes each token through a small subset of expert FFN blocks, so a model can have very large total parameters but small active parameters per token.

- Benefit: frontier-level quality at a fraction of the inference FLOPs.

- Cost: the full weight set must still be resident in memory, so self-hosting is VRAM-expensive even though it is compute-cheap.
- Practical impact: MoE is great for API-served models; for on-prem GPU-constrained deployments a dense 7-14B model is often the more realistic choice.

Q6. How do you evaluate whether a newer model is worth migrating to?

Never on vibes or on public leaderboards alone. My migration checklist:

- Run the frozen golden set (200-500 real production cases) against both models with identical prompts.
- Compare on task metrics, not preference: exact match, F1, groundedness, tool-call validity, refusal rate.
- Shadow-run the new model on live traffic without serving its output; diff the responses.
- Check breaking changes: tokenizer, function-calling schema, system-prompt sensitivity, safety behaviour.
- Canary 5% -> 25% -> 100% with an instant rollback flag.

Prompts are model-specific assets. Budget for prompt re-tuning in every migration - it is the step teams always forget.

Q7. Deterministic output is required by the business. What do you tell them?

I explain that LLMs are not deterministic systems and then engineer around it:

- Temperature 0 plus a fixed seed where supported reduces but does not eliminate variance (GPU floating-point non-determinism, batching effects, silent provider updates).
- Constrain the output surface: JSON schema / structured outputs, enums instead of free text, tool calls instead of prose.
- Move the deterministic part out of the model: let the LLM decide, let code execute. Calculations, thresholds and business rules belong in deterministic code.
- Cache: identical input returns the stored, already-validated output.
- Pin model versions explicitly and treat a version bump as a release.

Q8. Explain quantisation and its trade-offs.

Quantisation reduces weight precision (FP16 to INT8/INT4) to cut memory and increase throughput.

- FP16/BF16: baseline quality, highest VRAM.
- INT8: roughly 2x memory saving, near-lossless for most tasks.
- INT4 (GPTQ, AWQ, GGUF Q4): roughly 4x saving, noticeable degradation on reasoning and long-chain tasks; usually fine for classification and summarisation.

Architect view: quantise the model that serves the easy 80% of traffic, keep full precision for the escalation path. Always re-run evals after quantising - quality loss is task-dependent, not uniform.

Q9. What is the difference between a base model, an instruct model and a reasoning model?

- Base: raw next-token predictor. Good for fine-tuning, poor for direct use.
- Instruct/chat: aligned with SFT plus RLHF/DPO to follow instructions and maintain a conversation. This is the default production choice.
- Reasoning: trained to spend extra inference-time compute on internal deliberation before answering. Much better on maths, planning, multi-constraint problems; slower and more expensive.

Deploy reasoning models selectively - for planners in agent systems, complex root-cause analysis, or hard escalation paths, not for every call.

2. Prompt & Context Engineering

Q10. How do you structure a production system prompt?

I use a fixed skeleton so prompts stay diffable and testable:

- Role and scope: who the assistant is and what it must not do.
- Operating rules: hard constraints, escalation and refusal conditions.
- Tool contract: when to call which tool and what to do on failure.
- Output contract: exact schema or format, with an example.
- Retrieved context, clearly delimited and attributed.
- Task restatement at the very end (recency bias helps compliance).

The system prompt is source-controlled, versioned, and covered by regression evals. Treat it as code, not configuration.

Q11. Chain-of-thought, few-shot, ReAct - when do you use each?

- Few-shot: when the output format or edge-case handling is hard to describe but easy to demonstrate. 3-5 diverse examples, including a negative or edge case.
- Chain-of-thought: multi-step reasoning. In production, keep the reasoning hidden and return only the final structured answer to avoid leaking intermediate mistakes to users.
- ReAct (reason + act): whenever the model needs external tools; interleaves thought, action, observation.
- Self-consistency: sample n times and majority-vote. Expensive - reserve it for high-stakes, low-volume decisions.

Modern reasoning models make explicit CoT prompting largely redundant; over-prompting them can actually degrade results.

Q12. What is context engineering and why has it replaced 'prompt engineering' as the architect concern?

Prompt engineering optimises one string. Context engineering designs everything that occupies the window across a whole session:

- What is retrieved and how much of it.
- What is summarised or dropped as the conversation grows.
- What is stored in short-term versus long-term memory.
- Which tool results are kept verbatim versus compacted.

Practical techniques: rolling summarisation, sliding windows with pinned anchors, scratchpad files outside the context, sub-agent isolation so a noisy sub-task never pollutes the main thread, and just-in-time retrieval instead of pre-loading.

Q13. How do you stop prompt injection when untrusted content enters the context?

Assume any retrieved document, web page, email or tool output is hostile.

- Structural separation: wrap untrusted content in delimiters and state explicitly that content inside is data, never instructions.
- Least privilege: the model's tools should be scoped so that a successful injection still cannot do damage - read-only credentials, no raw shell, allow-listed domains.
- Human-in-the-loop for irreversible actions: payments, deletions, external sends.
- Output filtering: scan responses for exfiltration patterns, markdown image beacons, and injected URLs.
- Dual-model check: a cheap classifier screens retrieved content before it reaches the main model.

No prompt wording alone is a control. Injection is an architecture problem, mitigated by permissions, not by politeness.

Q14. How do you version and test prompts across environments?

- Prompts live in the repo (or a prompt registry) with semantic versions and an owner.
- Every prompt has a linked eval suite; CI blocks a merge if the regression score drops beyond a threshold.
- Environment config pins prompt version plus model version together, since the pair is what determines behaviour.
- A/B or shadow test in production, log prompt_version on every trace so any incident is traceable to an exact prompt.
- Keep a rollback path that does not require a redeploy.

Q15. Give a practical technique to reduce hallucination without changing the model.

Layered, cheapest first:

- Ground it: force the answer to cite retrieved chunk IDs and refuse when no chunk supports the claim.
- Give an explicit escape hatch: 'If the context does not contain the answer, reply exactly: INSUFFICIENT_CONTEXT.' Models hallucinate partly because they have no permitted way to fail.
- Constrain output to a schema so free-form invention has nowhere to go.
- Post-hoc verification: a second cheap call checks each claim against the source (NLI-style entailment).
- Show provenance in the UI so users can verify - this converts a silent failure into a visible one.

3. RAG Architecture

Q16. Walk me through a production RAG architecture end to end.

Two pipelines, not one.

- Ingestion (offline): source connectors -> parsing and OCR -> cleaning -> semantic chunking -> metadata enrichment (source, ACL, timestamp, section) -> embedding -> upsert to vector store plus keyword index. Incremental, idempotent, with change detection.
- Serving (online): query understanding and rewrite -> hybrid retrieval (dense + BM25) -> metadata and ACL filtering -> reranking -> context assembly with token budget -> generation with citation requirements -> guardrails -> response.
- Cross-cutting: caching, tracing, evaluation loop, feedback capture, and a re-index job when the embedding model changes.

The mistake I look for in candidates is treating RAG as 'embed and search'. The value is concentrated in chunking, hybrid retrieval, reranking and evaluation.

Q17. Your RAG returns plausible but wrong answers. How do you debug it?

Isolate retrieval from generation - always in that order.

- Step 1: is the correct chunk in the top-k at all? Measure recall@k against a labelled set. If not, it is a retrieval problem: fix chunking, embeddings, hybrid search or filters.
- Step 2: if it is retrieved but ranked low, add or tune a cross-encoder reranker.
- Step 3: if the right chunk is in context but the answer is still wrong, it is a generation problem: prompt grounding, citation enforcement, context ordering, or context too long.
- Step 4: check for conflicting or stale documents - duplicate versions of a policy are a very common root cause.

Instrument each stage separately. Teams that log only the final answer can never localise the failure.

Q18. What chunking strategy do you use and why?

There is no universal size; it is driven by document structure and query type.

- Structure-aware chunking first: split on headings, sections, table boundaries, or code blocks - not blind character counts.
- Typical ranges: 300-800 tokens for prose Q&A, larger for narrative synthesis, smaller for fact lookup.
- 10-20% overlap to avoid severing a sentence that carries the answer.
- Contextual retrieval: prepend a short LLM-generated summary of the parent document to each chunk. Large measured reduction in retrieval failures.
- Small-to-big: embed small precise chunks for matching, but pass the larger parent section to the LLM.
- Keep tables intact and serialise them to markdown; splitting a table is the single most common ingestion bug.

Q19. Explain hybrid search and reranking.

- Dense (vector) search captures semantics and paraphrase but fails on exact identifiers, error codes, part numbers and rare acronyms.
- Sparse (BM25/keyword) nails exact terms but misses synonyms.
- Hybrid runs both and fuses with Reciprocal Rank Fusion or weighted scores. This is the default I recommend for enterprise corpora, which are full of IDs and jargon.
- Reranking: retrieve broadly (top 50-100), then a cross-encoder scores each query-document pair jointly and returns the top 5-10. It is far more accurate than bi-encoder similarity and it is where most RAG quality gains come from cheaply.
- Cost control: reranking is per-candidate, so cap the candidate list and cache.

Q20. How do you enforce document-level security in RAG?

Never rely on the LLM to keep a secret.

- Store ACL identifiers as metadata on every chunk at ingestion.
- Filter at query time in the vector store using the caller's identity - pre-filter, not post-filter, so unauthorised vectors are never scored.
- For strict tenants, use physically separate indexes or namespaces per tenant.
- Propagate the end user's identity through the whole chain; do not let a service account see everything.
- Audit: log which chunk IDs were retrieved for which user - required for most enterprise security reviews.
- Re-sync permissions on a schedule; stale ACLs are a real leak vector when someone leaves a project.

Q21. How do you evaluate a RAG system?

Separate retrieval metrics from generation metrics.

- Retrieval: recall@k, precision@k, MRR, NDCG against a labelled query-to-chunk set.
- Generation: faithfulness/groundedness (is every claim supported by context), answer relevance, completeness, citation accuracy.
- End to end: task success rate, human thumbs, deflection rate, escalation rate.
- Tooling: RAGAS, TruLens, DeepEval, or a custom LLM-as-judge with a calibrated rubric.
- Build a golden set of 100-300 real queries early. Without it every change is guesswork.

Q22. When is RAG the wrong answer?

- When the task needs the whole corpus at once (global summarisation, 'what are the themes across all tickets') - use aggregation, map-reduce, or GraphRAG instead.
- When the requirement is behaviour or format, not knowledge - that is fine-tuning or prompting.
- When the data is structured and queryable - a text-to-SQL path over the warehouse is more accurate and auditable than embedding rows.
- When the corpus is tiny and stable - just put it in the prompt and use prompt caching.
- When freshness is measured in seconds - call the source API live.

Q23. What is GraphRAG and when would you use it?

GraphRAG builds an entity-relationship graph from the corpus and traverses it alongside vector search.

- Use it for multi-hop questions ('which suppliers are affected by a change at plant X that feeds product line Y'), where the answer is a relationship spanning several documents.
- Use it for global questions where community summaries over the graph beat top-k chunks.
- Cost: graph construction is LLM-heavy and maintenance is non-trivial. I recommend it only after standard hybrid RAG plus reranking has demonstrably plateaued.

4. Embeddings & Vector Stores

Q24. How do you select an embedding model?

- Start from MTEB but validate on your own domain data - domain shift is real, especially in engineering, legal and clinical corpora.
- Dimensionality vs cost: 768-1536 dims is the practical sweet spot. Matryoshka embeddings let you truncate dimensions and trade accuracy for storage.
- Multilingual requirement, max input length, and licence for self-hosting.
- Critical operational point: the embedding model is a hard dependency. Changing it invalidates the entire index and forces a full re-embed, so plan for blue-green indexes from day one.

Q25. Compare the main vector database options at an architectural level.

Option	Best for	Watch out for
pgvector / Postgres	Teams already on Postgres; transactional + vector in one place	Scaling past tens of millions of vectors needs care
OpenSearch / Elasticsearch	Hybrid search, existing ELK estate, strong filtering	Operational overhead; tuning kNN params
Pinecone / managed SaaS	Fast time to market, serverless scaling	Cost at scale, data residency, vendor lock-in
Qdrant / Weaviate / Milvus	Self-hosted control, rich filtering, on-prem	You own the ops and the upgrades
FAISS	Embedded, offline, prototypes	No persistence/ACL/multi-tenancy out of the box

My default recommendation for an enterprise starting out is pgvector or OpenSearch, because the operational cost of a new datastore usually outweighs the marginal recall gain.

Q26. Explain HNSW and the knobs that matter.

Hierarchical Navigable Small World is a graph-based approximate nearest neighbour index.

- M: connections per node. Higher M means better recall and more memory.
- ef_construction: build-time search width. Higher means a better graph and slower indexing.
- ef_search: query-time search width. The main runtime accuracy-vs-latency dial.
- Trade-off framing for interviews: ANN buys you sub-linear search at the cost of exactness. Always measure recall against exact search on a sample before signing off on parameters.

Q27. How do you handle updates, deletions and re-indexing?

- Idempotent upserts keyed by a stable document ID plus content hash so unchanged documents are skipped.
- Soft delete via a metadata flag filtered at query time, with periodic hard compaction.
- Tombstone propagation is the compliance-critical path - a deleted document must disappear from retrieval immediately, so I filter on delete first and compact later.
- Re-embedding: build a new index alongside the old one, dual-write, evaluate, then flip an alias. Never re-embed in place.
- Track index version in every trace so results can be explained after the fact.

5. Fine-tuning, Adaptation & Training

Q28. RAG vs fine-tuning vs prompt engineering - how do you decide?

Need	Approach
Fresh, changing, or citable knowledge	RAG
Consistent format, tone, or domain style	Fine-tuning
Behaviour achievable with instructions	Prompt engineering
Specialised task at lower cost/latency	Fine-tune a small model (distillation)
Complex reasoning over private data	RAG + strong base model

Order of attempt: prompt -> few-shot -> RAG -> fine-tune. Fine-tuning is the most expensive to maintain because every base-model upgrade re-opens the question. In practice most 'we need fine-tuning' requests are solved by better retrieval and a better output contract.

Q29. Explain LoRA and QLoRA in practical terms.

- LoRA freezes the base weights and trains small low-rank adapter matrices injected into attention/FFN layers. Typically under 1% of parameters are trained, checkpoints are megabytes not gigabytes, and you can hot-swap adapters per tenant or per task.
- QLoRA additionally loads the base model in 4-bit, so a 7-13B model fine-tunes on a single consumer or mid-range GPU.
- Key hyperparameters: rank r (8-64), alpha (often $2 \times r$), target modules, learning rate around $1e-4$ to $2e-4$, and 2-3 epochs.
- Architect angle: LoRA turns fine-tuning from a capex project into a routine pipeline, and adapter-per-client is a clean multi-tenant pattern.

Q30. What data do you actually need for a useful fine-tune?

- Quality dominates quantity: 500-2,000 high-quality, deduplicated, human-verified examples usually beat 50,000 scraped ones.
- Coverage of edge cases and refusals, not just the happy path - otherwise the model becomes confidently wrong at the boundaries.
- Held-out test split created before any training, never touched during iteration.
- Formatting must match inference exactly, including the system prompt.
- Governance: provenance, licensing, PII scrubbing and a documented consent basis. This is where enterprise fine-tuning projects usually stall, not on the GPUs.

Q31. What is catastrophic forgetting and how do you mitigate it?

Fine-tuning narrowly can degrade general capability - the model gets better at your task and worse at everything else.

- Use parameter-efficient methods (LoRA) so base weights stay intact.
- Mix in a slice of general instruction data during training.
- Lower learning rates and fewer epochs.
- Always evaluate on a general benchmark as well as your task set, and on safety behaviour - alignment is one of the first things to erode.

Q32. Explain distillation as a cost strategy.

Use a frontier model to generate high-quality labelled outputs, then fine-tune a small model on them.

- Typical result: 80-95% of the quality at 10-20x lower inference cost and much lower latency.
- Best for narrow, high-volume tasks: classification, routing, extraction, test-case generation, log triage.
- Check the provider's terms of service on training from outputs before you build a business case on it.
- Keep the teacher model as the fallback for low-confidence cases - a cascade, not a replacement.

Q33. What is RLHF/DPO and would you ever do it in an enterprise?

- RLHF trains a reward model on human preference pairs, then optimises the policy against it. DPO achieves similar alignment directly from preference pairs without a separate reward model, and is far simpler and cheaper.
- Enterprise reality: rarely worth it. You need a serious preference-collection operation. In almost all cases, SFT/LoRA plus good prompting and guardrails is the right cost-benefit point.
- Where it does appear: tuning a model to an organisation's tone-of-voice or risk posture at scale, usually via DPO on curated pairs collected from production feedback.

6. Agentic AI & Multi-Agent Systems

Q34. What makes a system 'agentic', and when should you avoid agents?

An agent is an LLM in a loop with tools, memory and autonomy over control flow: it decides what to do next rather than following a fixed graph.

- Use an agent when the number of steps is unknown in advance and depends on intermediate results.
- Use a deterministic workflow (a DAG with LLM nodes) when the steps are known - it is cheaper, faster, testable and debuggable.
- Avoid agents when errors are irreversible, latency budgets are tight, or the task can be expressed as three chained prompts.

The most valuable architect instinct here is restraint: most 'agent' requirements are actually workflow requirements with one or two LLM decision points.

Q35. Describe the main agent design patterns.

- ReAct loop: think, act, observe, repeat until done. The workhorse pattern.
- Planner-executor: one model decomposes the goal into a plan, cheaper models execute steps. Good for cost control.
- Reflection / critic: a second pass reviews and revises the output. Strong quality gains on writing and code.
- Router / dispatcher: classify the request and dispatch to a specialised sub-agent or workflow.
- Orchestrator with sub-agents: each sub-agent has an isolated context window and returns a compact summary, protecting the main context.
- Evaluator-optimiser: generate, score against a rubric, regenerate until the score passes or attempts are exhausted.

Q36. How do you keep a long-running agent from going off the rails?

Bound it in every dimension:

- Hard limits: max iterations, max tool calls, max tokens, max wall-clock time, max spend per task.
- Loop detection: hash the state or the last n actions and abort on repetition.
- Progress checks: every k steps, force the agent to restate the goal and what remains; terminate if there is no measurable progress.
- Structured termination: an explicit 'finish' tool the agent must call, so ending is a deliberate action rather than a fade-out.
- Human checkpoints on irreversible or high-value actions.
- Full trace logging so any run can be replayed step by step.

Q37. How do you handle errors and tool failures inside an agent loop?

- Return errors to the model as structured, actionable observations ('400: field date must be ISO-8601') rather than raw stack traces - the model can often self-correct.
- Retry with exponential backoff for transient failures; do not let the LLM retry a deterministic 4xx.
- Circuit-break a tool after repeated failures and tell the agent the tool is unavailable so it can find another route or stop cleanly.
- Make tools idempotent, or use idempotency keys, because agents will re-call them.
- Compensating transactions (a saga pattern) for multi-step external side effects.
- Always have a defined 'give up and escalate to a human' path with the context attached.

Q38. Single agent with many tools, or many specialised agents?

Start single. Split only when there is a concrete reason:

- Split when the tool count exceeds what fits reliably in one prompt (practically, when selection accuracy starts dropping - often past 15-25 tools).
- Split when domains need different system prompts, models, or permissions.
- Split when context isolation matters, so a noisy sub-task cannot pollute the main thread.

Costs of splitting: orchestration complexity, latency from extra hops, information loss between agents, and much harder debugging. Multi-agent is a scaling answer, not a default architecture.

Q39. What is MCP and why does it matter architecturally?

The Model Context Protocol is an open standard that lets models connect to tools, data sources and prompts through a uniform client-server interface.

- Before MCP: every model-tool pairing was bespoke, an $M \times N$ integration problem.
- With MCP: build a server once, any compliant client can use it - $M + N$.
- Primitives: tools (model-invoked actions), resources (data the client can read), prompts (reusable templates).
- Architect value: it turns integrations into reusable, independently governed assets, and gives you one place to enforce auth and scoping.
- Risks to control: server supply chain, over-broad scopes, tool-description injection, and token bloat from too many registered tools.

Q40. How do you design tools so an LLM actually uses them correctly?

Tool design is prompt design.

- Names describe intent, not implementation: 'find_open_defects_by_release', not 'query_db_2'.
- Descriptions state when to use AND when not to use the tool.
- Strict typed schemas with enums; make invalid calls unrepresentable.
- Few, coarse-grained tools beat many fine-grained ones. Consolidate multi-step API sequences into one purposeful tool.
- Return compact, model-readable results - filtered fields, not raw 4,000-line JSON. Token bloat in observations is a top cause of agent failure.
- Include an example call in the description for anything ambiguous.

Q41. How do you handle memory in an agent system?

- Short-term: the conversation window, managed with rolling summarisation and pinned anchors (goal, constraints, decisions made).
- Working memory: an external scratchpad or file the agent reads and writes, keeping bulk state out of the context.
- Long-term: a vector or key-value store of durable facts and preferences, retrieved just-in-time rather than pre-loaded.
- Episodic: past run traces used for few-shot examples of what worked.
- Governance: memory needs a write policy, a retention policy, a privacy filter and a user-visible way to correct or delete it. Unbounded memory is both a cost and a compliance problem.

Q42. How do you evaluate an agent, as opposed to a single LLM call?

- Final outcome: task success rate against a checkable end state - the primary metric.
- Trajectory quality: were the right tools called, in a sensible order, without redundant steps?
- Efficiency: steps, tokens, wall-clock time and cost per successful task.
- Robustness: behaviour under injected tool failures and adversarial inputs.

- Simulation environments with seeded state so runs are reproducible.

The trap is judging only the final answer. An agent that succeeds through a wasteful, lucky path will fail at scale, so trajectory metrics predict production behaviour better.

7. LLMOps, Deployment & Infrastructure

Q43. What does a reference production LLM platform look like?

Layered, so each concern can change independently:

- Client layer: apps, chat surfaces, IDE plugins, CI hooks.
- Gateway layer: authentication, per-tenant rate limits and quotas, routing and fallback, prompt caching, cost attribution, PII redaction, audit logging. This is the single most valuable component to build first.
- Orchestration layer: workflows, agents, tool registry, memory.
- Knowledge layer: ingestion pipelines, vector and keyword indexes, document store.
- Model layer: hosted APIs plus self-hosted endpoints behind a common interface.
- Observability and governance: tracing, evals, feedback, guardrails, model registry, policy enforcement.

The gateway is what stops every team from re-implementing auth, retries and cost control - and it is what makes model swaps a config change.

Q44. How do you decide between managed APIs and self-hosting?

Factor	Managed API	Self-hosted
Time to value	Days	Weeks to months
Cost profile	Variable, per token	Fixed GPU capex/opex; cheaper at very high steady volume
Data control	Contractual	Full, air-gappable
Model quality	Frontier available	Open weights, typically a step behind
Ops burden	Low	High: GPUs, scaling, upgrades, on-call

The break-even is usually about GPU utilisation. Self-hosting only wins if you can keep the accelerators busy; a 20%-utilised H100 fleet is far more expensive than an API. Regulated data, however, can override the economics entirely.

Q45. How do you scale self-hosted inference?

- Serving stack: vLLM, TGI, TensorRT-LLM or SGLang - with continuous (in-flight) batching, which is the single biggest throughput lever.
- PagedAttention for efficient KV-cache memory use, and prefix caching for shared system prompts.
- Separate prefill and decode pools for very different workloads.
- Autoscaling on queue depth and time-to-first-token, not CPU. Keep a warm pool because cold-starting a model is minutes.
- Quantisation and tensor parallelism to fit the model and raise throughput.
- Capacity planning by concurrent requests and KV-cache memory, not by request count alone.

Q46. What are your key latency levers?

- Stream tokens - perceived latency is dominated by TTFT.
- Prompt caching for stable system prompts and shared context (large cost and TTFT savings).
- Semantic caching for repeated or near-duplicate queries.
- Right-size the model per step; use a small model for routing and extraction.
- Parallelise independent retrieval and tool calls instead of chaining them.
- Cap output length - decode is usually the dominant cost.
- Speculative decoding where supported.

- Move guardrail checks in parallel with generation, not strictly before it, where policy allows.

Q47. How do you control and forecast cost?

- Measure cost per successful outcome, not cost per call. That reframes the whole conversation with finance.
- Attribute every call with tenant, feature, prompt version and model - no attribution means no optimisation.
- Set hard budgets and per-tenant quotas at the gateway, with graceful degradation to a cheaper model rather than an outage.
- Cascade: cheap model first, escalate on low confidence.
- Compress context aggressively - input tokens are usually the silent majority of spend in RAG systems.
- Cache at three levels: exact, semantic and prompt-prefix.
- Batch offline workloads to batch-API pricing where available (often around 50% cheaper).

Q48. What does a CI/CD pipeline for an LLM application include that a normal one does not?

- Prompt and eval versioning alongside code.
- An automated regression eval gate on the golden set - build fails if faithfulness or task success drops beyond a threshold.
- Adversarial and safety test suites (injection, jailbreak, PII leakage) run on every release.
- Model-version pinning and a documented rollback of prompt + model together.
- Index/embedding migration steps as a first-class deployment artefact.
- Canary release with live quality metrics, not just error rates, because an LLM regression is usually silent - the service returns 200 with a worse answer.

Q49. How do you make an LLM service resilient?

- Multi-provider fallback behind the gateway, with prompts validated against each fallback model in advance.
- Timeouts, retries with jitter, and circuit breakers per provider.
- Queue and backpressure for bursty traffic; degrade to async 'we'll email you the answer' rather than fail.
- Graceful degradation ladder: frontier model -> smaller model -> cached answer -> deterministic template -> human handoff.
- Chaos-test the failure paths; provider outages are frequent enough to plan for.

8. Evaluation, Observability & Quality

Q50. How do you build an evaluation strategy from scratch?

- Start by defining what 'good' means with the business, in measurable terms, before writing any prompt.
- Build a golden set of 100-300 real cases with expected outcomes; grow it from production failures, so every incident becomes a permanent test.
- Layer the evals: deterministic checks (schema valid, contains citation, latency budget) -> statistical metrics -> LLM-as-judge for subjective quality -> human review on a sample.
- Run the suite in CI and on a live production sample.
- Track drift over time and alert on it.

Golden sets are the highest-leverage artefact in an AI programme. Teams without one cannot answer 'did that change help?'

Q51. How reliable is LLM-as-a-judge, and how do you make it trustworthy?

Useful and scalable, but it must be validated like any other measurement instrument.

- Calibrate against human labels; report agreement (e.g. Cohen's kappa). If the judge does not agree with humans, the score is noise.
- Use a detailed rubric with concrete anchors, not 'rate 1-10'.
- Prefer pairwise comparison over absolute scoring - it is more stable.
- Control known biases: position bias (randomise order), verbosity bias, and self-preference bias (avoid judging a model with itself where possible).
- Use a strong model as judge even if a weaker one serves production.

Q52. What do you instrument in production?

- Full trace per request: prompt version, model version, retrieved chunk IDs, tool calls and arguments, tokens in/out, latency per stage, cost, and the final output.
- Quality signals: user thumbs, edit distance between the suggestion and what the user shipped, escalation rate, task completion, regeneration rate.
- Safety signals: guardrail triggers, refusal rate, PII detections, injection detections.
- Business KPIs: deflection, cycle-time saved, defect escape rate - whatever the project was funded on.

Tooling: OpenTelemetry-based tracing plus an LLM observability layer (LangSmith, Langfuse, Arize, Phoenix). The rule is that any production answer must be fully reconstructable months later.

Q53. How do you detect drift when neither the model nor the prompt has changed?

Drift comes from the world, the data and the users:

- Input drift: monitor embedding distributions of incoming queries and alert on distance from the baseline.
- Corpus drift: documents change; track index freshness and retrieval score distributions.
- Silent provider drift: run a canary eval set on a schedule against the live endpoint to catch unannounced model updates.
- Outcome drift: track task success and user feedback weekly with control limits.
- Alert on trend, not single points, to avoid pager fatigue.

Q54. How do you measure ROI on an AI initiative to defend the budget?

- Baseline first: measure the manual process before deployment - cycle time, cost per case, error rate. Without a baseline there is no ROI story.
- Define the value metric: hours saved, deflection rate, defects caught earlier, revenue influenced.

- Attribute honestly with a control group or a staged rollout.
- Total cost: inference plus engineering plus review overhead plus infrastructure.
- Report adoption alongside quality - an accurate system nobody uses has zero ROI, and this is the most common real-world failure mode.

9. Security, Governance & Responsible AI

Q55. Walk through the top LLM security risks and your controls.

- Prompt injection (direct and indirect): isolate untrusted content, least-privilege tools, human approval for irreversible actions, output scanning.
- Sensitive information disclosure: PII detection and redaction on input and output, ACL-filtered retrieval, no secrets in prompts.
- Supply chain: pin and scan models, adapters, MCP servers and libraries; verify checksums and provenance.
- Insecure output handling: never pass model output to eval(), a shell, or raw SQL; treat output as untrusted user input and sandbox all execution.
- Excessive agency: scope tokens narrowly, per-tool permissions, spend and rate caps.
- Unbounded consumption: quotas, max token limits, cost circuit-breakers.
- Data poisoning: validate and version training and RAG corpora with provenance.

Q56. How do you design guardrails?

Defence in depth, at three points:

- Input: PII/secret detection, injection classifiers, topic and jurisdiction policy, rate limits.
- In-flight: system-prompt constraints, structured output schemas, tool permission checks at call time (authorise in code, not in the prompt).
- Output: toxicity and policy classifiers, groundedness check against retrieved context, PII scrub, regex checks for secrets and internal URLs.
- Fail-safe behaviour: on guardrail failure, degrade to a safe canned response plus human escalation rather than a raw error.
- Log every trigger and review them - guardrail logs are the best source of new eval cases.

Q57. How do you handle PII and data residency in an LLM pipeline?

- Classify data at ingestion and tag every chunk with sensitivity and jurisdiction.
- Minimise: redact or tokenise PII before it reaches the model, and re-hydrate afterwards where the use case requires it.
- Regional deployment and regional model endpoints; enforce residency at the gateway routing layer.
- Zero-retention or no-training contractual terms with providers, verified in writing.
- Retention policy on prompts, traces and memory - logs are the most commonly overlooked PII store in AI systems.
- Right to erasure must propagate to the vector index, caches and logs, not only the source database.

Q58. What does AI governance look like in practice, and how do you map to the EU AI Act or the NIST AI RMF?

- Inventory: a registry of every AI use case, its owner, model, data sources and risk tier. You cannot govern what you have not catalogued.
- Risk tiering: EU AI Act classifies by risk (unacceptable / high / limited / minimal). High-risk uses (employment, credit, essential services, biometrics) carry obligations on data governance, documentation, human oversight, robustness and logging.
- NIST AI RMF gives the operating functions: Govern, Map, Measure, Manage - useful because it is voluntary and maps cleanly onto engineering practice.
- Artefacts: model cards, system cards, DPIAs, eval reports, incident procedures.
- Human oversight designed in, with a defined reviewer and an override path.

- Transparency: disclose AI involvement to end users where required.

Practical advice: build the evidence as a by-product of the pipeline (traces, evals, approvals) rather than as an annual documentation exercise.

Q59. How do you approach bias and fairness testing?

- Define the protected attributes and the harm in context - fairness is task-specific, not generic.
- Build counterfactual test sets: hold everything constant and vary names, genders, dialects, locations; measure output deltas.
- Measure outcome disparities across cohorts, not just intent.
- Watch the retrieval layer too - biased corpora produce biased grounded answers that look well-sourced.
- Document limitations honestly in the model card and keep a human review path for consequential decisions.

Q60. Who is accountable when an AI system gives harmful advice?

The deploying organisation. That is the answer interviewers want, and it should shape the architecture:

- Keep humans accountable for consequential decisions; the system recommends, a person decides.
- Maintain a complete audit trail so any output can be reconstructed and explained.
- Define scope explicitly and refuse out-of-scope requests rather than degrading silently.
- Have an incident response runbook for AI failures, including kill-switch, rollback, user notification and root-cause analysis.
- Insurance, legal review and contractual terms with model providers belong in the design conversation, not after the incident.

10. AI for Quality Engineering & Testing

Q61. How does AI change a QA strategy in practice?

- Requirement analysis: LLMs generate test scenarios and edge cases from user stories and acceptance criteria, surfacing ambiguity in the requirement itself - the earliest and cheapest defect removal point.
- Test design: generate boundary, negative and combinatorial cases; convert manual test cases into automation code.
- Self-healing automation: when a locator breaks, a model proposes a repaired selector from the DOM and visual context, cutting maintenance drastically.
- Defect triage: cluster and deduplicate defects, propose severity and route to the right team.
- Risk-based selection: predict which tests to run for a given change set, compressing regression cycles.
- Log and failure analysis: summarise failed runs and suggest probable root cause.

The strategic shift is from writing tests to curating and governing generated tests. Volume stops being the constraint; relevance and maintenance become the constraint.

Q62. Design an agentic QA platform for an enterprise application.

Layered agent architecture with deterministic execution:

- Requirement agent: parses stories and acceptance criteria into a structured, traceable test-intent model.
- Test design agent: generates scenarios with coverage rationale and links each back to a requirement ID.
- Script generation agent: emits framework-native code (Playwright, Selenium, RestAssured) against a page-object or API contract layer.
- Execution layer: deterministic CI runners, parallel grid, no LLM in the hot path.
- Analysis agent: triages failures into product defect, test defect, environment or flake.
- Maintenance agent: proposes locator and assertion repairs as pull requests, never auto-merged.
- Governance throughout: human approval gates, full traceability from requirement to test to defect, and eval metrics on generated-test quality.

The single most important design rule: the LLM designs and repairs, deterministic code executes. Otherwise your test results become as non-deterministic as the model.

Q63. How do you validate that AI-generated tests are actually good?

Generated volume is worthless without quality control.

- Mutation testing: inject faults into the application and measure whether the generated suite catches them. This is the strongest available signal.
- Requirement traceability coverage: every acceptance criterion maps to at least one test.
- Duplicate and redundancy detection via embedding similarity across the suite.
- Flakiness quarantine: run new tests n times before admitting them to the main suite.
- Defect escape rate over time as the ultimate outcome metric.
- Human review gate for the first releases, sampled thereafter.

Q64. How do you test a non-deterministic AI feature itself?

Traditional assertions do not apply, so shift from exact match to property and distribution testing.

- Property-based assertions: the output must be valid JSON, must cite a retrieved source, must not contain PII, must stay within a length bound.
- Semantic similarity to a reference answer with a threshold, rather than string equality.
- LLM-as-judge against a rubric, calibrated to human labels.
- Statistical testing: run n times and assert a pass rate (e.g. 95 of 100), not a single pass.

- Adversarial suites: injection, jailbreak, ambiguous and out-of-scope inputs.
- Regression via golden sets with score thresholds in CI.
- Freeze randomness where possible: temperature 0, pinned model version, seeded data.

Q65. How do you use AI for test data management?

- Synthetic data generation that preserves referential integrity and realistic distributions, avoiding production PII entirely.
- Schema-aware generation from the data model so constraints and foreign keys hold.
- Edge-case seeding: deliberately generate boundary and malformed records that manual data sets never include.
- Masking and tokenisation pipelines where production-like data is unavoidable, with consistent mapping across systems.
- Validation: statistical comparison of synthetic versus production distributions so tests remain representative.

11. Cloud, Data & Integration Architecture

Q66. Compare the main cloud AI platforms at an architect level.

Platform	Strengths	Typical fit
AWS Bedrock / SageMaker	Multi-model choice, deep AWS integration, private networking	Enterprises already standardised on AWS
Azure AI Foundry / Azure OpenAI	Enterprise identity, compliance posture, Microsoft 365 and Fabric integration	Regulated enterprises on the Microsoft stack
Google Vertex AI	Gemini, strong data and BigQuery integration, grounding with search	Data-heavy and analytics-led organisations
Databricks / Snowflake	Model serving next to the governed data, no data movement	Where the lakehouse is the centre of gravity

The architect answer is that the decision is usually driven by where the data and the identity provider already live, not by marginal model quality - because data gravity and compliance dominate.

Q67. How do you design the data pipeline that feeds an enterprise AI system?

- Connectors with change data capture for incremental sync, not full re-crawls.
- Parsing tier that handles PDFs, scanned images (OCR), tables, and office formats - document parsing quality sets the ceiling on RAG quality.
- Normalisation and deduplication, with a canonical document ID and content hash.
- Enrichment: metadata, ACLs, entities, effective dates, and document lineage.
- Quality gates: reject empty or garbled extractions before they poison the index.
- Orchestration with Airflow/Dagster, idempotent, replayable, and observable.
- A dead-letter queue and a re-processing path, because a percentage of documents always fail.

Q68. How do you integrate an AI capability into existing enterprise systems?

- Expose the capability as a versioned API behind the gateway; never let business systems call model providers directly.
- Event-driven where possible: publish results to a topic so downstream systems subscribe rather than poll.
- Asynchronous by default for anything over a couple of seconds, with webhooks or polling endpoints.
- Idempotency keys, because integrations retry.
- Backward-compatible contracts: output schemas are public interfaces and must be versioned.
- Feature flags per business unit so rollout is controlled and reversible.
- Plan the human-in-the-loop UI as part of the integration, not as an afterthought - review workflows are where adoption is won or lost.

Q69. Text-to-SQL over the warehouse: how do you make it safe and accurate?

- Restrict the surface: read-only role, curated semantic layer or views, row-level security enforced by the database, not the model.
- Give the model a rich schema context: column descriptions, sample values, known joins, business definitions of metrics.
- Few-shot with verified query pairs from your actual analysts.
- Validate before execution: parse the SQL, block DDL/DML, enforce a LIMIT and a query cost/timeout ceiling.
- Execute, then have the model explain the result and show the SQL for transparency.
- Evaluate with execution-accuracy on a labelled question-to-SQL set, not string match.

- Cache validated question-to-SQL pairs; over time the system answers most questions from the cache.

12. System Design Case Studies

Q70. Design an enterprise knowledge assistant for 50,000 employees over SharePoint, Confluence and ServiceNow.

I would answer in five moves: clarify, size, design, trade off, evolve.

- Clarify: who are the users, what is the accuracy bar, what is the latency budget, what are the permission rules, and what does success mean (deflection? time saved?).
- Size: assume 50k employees, 10% daily active, 5 queries each = 25k queries/day, peaking around 10-15 QPS. Corpus perhaps 5M documents, 30M chunks. This is modest - the hard part is permissions and freshness, not scale.
- Design: incremental connectors with CDC -> parsing and chunking -> embeddings -> vector plus keyword index with ACL metadata; serving path does query rewrite, hybrid retrieval with pre-filtering on the caller's groups, reranking, generation with mandatory citations, guardrails, streaming response.
- Trade-offs: hybrid retrieval and reranking cost latency but are needed for enterprise jargon; per-user ACL filtering rules out a shared cache of answers, so I cache at the embedding and prompt-prefix level instead.
- Evolve: start with the top 3 content sources and one department, measure deflection and groundedness, then expand. A phased rollout also builds the golden set.

Risks I would raise unprompted: stale and contradictory documents, permission drift, and low adoption if answers are not visibly cited.

Q71. Design an AI system that triages and resolves production incidents.

- Ingest: alerts, logs, metrics, traces, recent deployments, and change tickets.
- Correlate: cluster related alerts into a single incident; retrieve similar historical incidents and their resolutions from a vector store.
- Diagnose: an agent with read-only tools (query logs, fetch metrics, diff the last deploy) proposes ranked hypotheses with evidence.
- Act: strictly tiered autonomy - read-only diagnostics fully automated; low-risk remediation (restart, scale out) behind one-click approval; anything else advisory only.
- Communicate: draft the status update and the postmortem timeline automatically.
- Learn: every resolved incident is written back as a labelled example.

Key design point: incident response is exactly where 'excessive agency' hurts. I would deliberately keep write actions human-approved until the diagnostic accuracy is proven over months.

Q72. Design a multi-tenant AI SaaS platform.

- Isolation: separate vector namespaces or indexes per tenant; tenant ID enforced at the data layer, never derived from the prompt.
- Configuration: per-tenant prompts, model choice, guardrail policy and tool permissions, stored as versioned config.
- Customisation: per-tenant LoRA adapters or per-tenant RAG corpora rather than separate base models.
- Cost: metering and quota per tenant at the gateway; noisy-neighbour protection with rate limits and priority queues.
- Compliance: per-tenant data residency routing and retention policy; per-tenant audit export.
- Evaluation: per-tenant golden sets, because quality is tenant-specific.

The classic failure is a shared cache leaking one tenant's answer to another - cache keys must include tenant and permission scope.

Q73. Design a document processing pipeline for 100,000 invoices a day.

- Do not use an LLM for everything. Layer it: classification (cheap model or classical ML) -> OCR/layout extraction (specialised document model) -> LLM only for the ambiguous residue.
- Batch: asynchronous queue-based processing with batch-API pricing; latency budget is hours, not milliseconds, so optimise purely for cost.
- Structured output with a strict schema and field-level confidence.
- Confidence routing: high confidence auto-posts, medium goes to a human review queue with the extracted fields pre-filled, low is fully manual.
- Feedback loop: reviewer corrections become training data; the human-review threshold falls as accuracy rises. This is the mechanism that makes the business case improve over time.
- Measure straight-through-processing rate as the headline KPI, plus cost per document and field-level accuracy.

Q74. Design a real-time voice AI agent for a contact centre.

- Pipeline: streaming ASR -> intent and context retrieval -> LLM -> streaming TTS, with barge-in support so the caller can interrupt.
- Latency is the entire design constraint: target under 800ms end to end. That means streaming at every hop, a small fast model or a speech-to-speech model, and pre-warmed connections.
- Overlap work: start retrieval while ASR is still finalising; begin TTS on the first sentence rather than the full response.
- Keep business logic deterministic: the LLM handles conversation, tools handle account lookups and transactions, with strict authentication before any account action.
- Fallback: confidence thresholds and explicit escalation to a human with full transcript context.
- Compliance: consent and recording disclosure, PII redaction in transcripts, and retention limits.

Q75. Design an AI coding assistant platform for 500 developers.

- Context is the differentiator: repository indexing, symbol graphs, and retrieval of the actual conventions in the codebase rather than generic completions.
- Guardrails: secret scanning on both prompt and output, licence checking on generated code, and no proprietary code sent to non-approved endpoints.
- Integration: IDE plus CI - review agents on pull requests are often higher value than autocomplete, because they catch issues rather than create volume.
- Governance: an allow-listed model set, audit logs, and a policy on what may be pasted.
- Measurement: acceptance rate, edit distance on accepted suggestions, PR cycle time, defect escape rate. Lines of code generated is a vanity metric.
- Change management: enablement and prompting standards matter more than the tool choice; adoption curves are the real risk.

13. Architecture Trade-off Questions

Q76. Latency versus accuracy - how do you decide?

Frame it as a product decision, backed by measurement:

- Quantify the accuracy gain of each expensive step (reranking, self-consistency, reflection) on the golden set.
- Quantify the latency and cost each adds.
- Map to user tolerance: interactive chat needs fast first tokens; a nightly batch report can take a minute.
- Then design tiers rather than a single compromise: fast path for the common case, thorough path when confidence is low or stakes are high.

Confidence-based routing is usually the answer - it lets you have both, most of the time.

Q77. Build versus buy for an AI capability?

- Buy the commodity: model serving, observability, vector storage, guardrail classifiers. These are undifferentiated and mature.
- Build what encodes your domain: the retrieval corpus, evaluation sets, prompt library, tool definitions and workflow logic. That is the defensible asset.
- Consider exit cost: how hard is it to migrate off the vendor? Keep an abstraction layer at the model and vector-store boundaries.
- Consider time to value against a fast-moving field - a 12-month build often ships into a world where the problem has been commoditised.

Q78. Monolithic prompt chain versus microservice-style decomposition?

- Decomposition gives independent scaling, targeted model selection per step, easier evaluation of each stage and clearer failure isolation.
- It costs latency (network hops), complexity, and information loss between stages.
- My rule: decompose along evaluation boundaries. If you cannot measure a step independently, do not split it into a service.
- Start with a modular monolith - separate modules, one deployable - and extract services only when scaling or ownership demands it.

Q79. When do you push work out of the LLM and into deterministic code?

Whenever the operation has a correct answer that code can compute.

- Arithmetic, date logic, sorting, filtering, threshold rules, regulatory calculations - all deterministic code.
- The LLM's job is interpretation, planning and language; the system's job is execution and verification.
- This dramatically improves testability and is usually the fastest route to both accuracy and cost improvement.

A useful line for interviews: 'I use the model for judgement, not for arithmetic.'

14. Leadership, Stakeholders & Behavioural

Q80. A senior stakeholder wants to 'add AI' to a process where it clearly will not help. How do you handle it?

- Start by understanding the underlying goal, not the requested solution - usually it is cost, speed or a competitive fear.
- Re-frame in terms of outcomes and propose measuring the current process first.
- Show the fit honestly: where AI adds value (unstructured input, judgement, volume) versus where rules or better data engineering win.
- Offer a cheap, time-boxed proof with a clear kill criterion. A two-week spike with a defined failure threshold de-risks the conversation and preserves the relationship.
- If it still proceeds against the evidence, document the risks and the assumptions in writing and design the smallest reversible version.

Architects earn credibility by saying no with data and an alternative, not by saying no with an opinion.

Q81. How do you get an AI POC into production - what usually blocks it?

Most POCs die for non-technical reasons. I plan for them from day one:

- No baseline or success metric, so nobody can prove value.
- Security and compliance review discovered late - I engage them in week one, not at go-live.
- Data access and permissions, which take longer than the build.
- No owner for ongoing costs and maintenance.
- Quality that was fine on 20 demo cases and collapses on the long tail - hence the golden set.
- No change management, so users never adopt it.

My standard approach is to build the POC on production-shaped rails: real auth, real data governance, real evals, small scope.

Q82. How do you upskill a traditional engineering team to work on AI systems?

- Start with mental models, not tools: non-determinism, evaluation-driven development, context as a scarce resource.
- Pair experienced engineers with the AI work rather than creating an isolated AI team - isolation kills adoption.
- Establish shared assets early: a prompt library, an eval harness, and reference architectures so people are not each inventing patterns.
- Set guardrails so experimentation is safe: sandbox keys, spend limits, approved model list.
- Measure learning by shipped, evaluated features, not by course completions.

Q83. Tell me about an AI project that failed or underdelivered. What did you learn?

Structure this with STAR and be specific about the diagnosis:

- Situation and task: the business goal and constraints.
- Action: what you built and, critically, what you assumed.
- Result: the honest outcome with numbers.
- Learning: the systemic change you made - for instance, 'we now require a labelled golden set and a measured baseline before any AI project is funded.'

Interviewers are testing self-awareness and whether the lesson became a process change. Avoid blaming the model or the vendor; own the architectural assumption that proved wrong.

Q84. How do you stay current in a field that changes monthly?

- Separate signal from noise: model releases matter less than pattern shifts (tool use, long context, reasoning models, MCP-style standards).
- Read primary sources - model cards, provider engineering blogs, and a small number of papers - rather than social summaries.
- Maintain a personal benchmark: a fixed set of your own hard tasks that you run against every significant new model. That converts hype into evidence.
- Build something small with each major new capability; hands-on beats reading.
- Keep an architecture decision log so you know which decisions to revisit when a constraint disappears.

Q85. How do you decide what NOT to build?

- Anything the platform vendor will ship in six months.
- Anything without a named owner and a measurable outcome.
- Anything where the failure mode is unacceptable and cannot be bounded by human review.
- Anything that requires data you cannot legally or practically obtain.
- Anything whose value depends on 99% accuracy from a system you can only get to 85%.

Saying no early is the highest-leverage thing an architect does, because AI projects consume disproportionate organisational attention.

15. Rapid-Fire Round

Q86. Quick answers to the short questions interviewers use as filters.

- Why does RAG beat long context for enterprise knowledge? Cost, freshness, citability, access control and stable retrieval quality - long context pays the token cost on every call and offers no permission model.
- What is prompt caching? Reusing the computed KV cache for a stable prompt prefix, cutting TTFT and input cost substantially for repeated system prompts.
- What is a cross-encoder? A model that scores a query and document jointly for accurate reranking, as opposed to a bi-encoder that embeds them separately for fast retrieval.
- What is function calling? The model returns a structured request to invoke a defined tool; your code executes it and returns the result as an observation.
- What is structured output / constrained decoding? Forcing generation to conform to a JSON schema or grammar at decode time, which eliminates parse failures.
- What is a KV cache? Stored attention keys and values for previous tokens so decoding does not recompute them; it dominates GPU memory in long-context serving.
- What is speculative decoding? A small draft model proposes tokens that the large model verifies in parallel, cutting latency without changing the output distribution.
- What is semantic caching? Returning a stored answer when a new query is close enough in embedding space to a previous one; needs a carefully tuned threshold and tenant-scoped keys.
- What is model distillation? Training a smaller model on a larger model's outputs to retain most of the quality at a fraction of the cost.
- What is an embedding drift problem? When the embedding model changes, old and new vectors are incomparable, so the whole index must be rebuilt.
- What is the difference between recall@k and precision@k? Recall@k is how much of the relevant material you retrieved; precision@k is how much of what you retrieved was relevant. RAG usually optimises recall then fixes precision with a reranker.
- What is groundedness/faithfulness? Whether every claim in the answer is supported by the retrieved context - the primary anti-hallucination metric.
- What is a golden dataset? A curated, frozen set of representative cases with expected outcomes, used as the regression gate for every change.
- What is human-in-the-loop versus human-on-the-loop? In-the-loop approves each action; on-the-loop supervises and can intervene. Choose by reversibility and blast radius.
- What is the biggest cause of AI project failure? Not model quality - it is the absence of a measurable baseline, clean data access, and user adoption.

Q87. Questions worth asking your interviewer.

- How do you currently measure the quality of your AI systems in production, and who owns that metric?
- What is the split between building on managed APIs and self-hosting, and what drove it?
- How mature is the data platform that these AI systems depend on?
- Where does AI governance sit organisationally, and how early does it engage?
- What has been the hardest AI system to get from POC into production here, and why?
- How is the architect role balanced between hands-on building and stakeholder work?

These signal that you think about operations and organisational reality, not just models - which is precisely the difference between an engineer and an architect.